시스템 분석 및 설계

02주차. 프로세스, 방법, 원리

서울사이버대학교

학습목표

1. 소프트웨어 각 개발 단계에서 생성해야 할 산출물이 무엇인지 말할 수 있다.
2. 구조적 방법론, 정보공학 방법론, 객체지향 방법론의 개요와 차이점을 기술할 수 있다.
3. 논리적 설계와 물리적 설계의 차이에 대해서 설명할 수 있다.
4. 소프트웨어 개발에 자주 사용되는 다이어그램을 이해하고 그릴 수 있다.

- 대부분의 IT조직은 비즈니스 목표를 달성하고 소프트웨어 제품 품질을 향상시킬 목적으로 저마다 표준 소프트웨어 개발 프로세스를 보유
- 같은 폭포수 모델을 사용하는 조직이라 하더라도 각 단계의 이름과 작업 절차에 차이를 가지는 경우가 대부분
- 그러나 소프트웨어 개발 절차에서 작업 절차와 명칭을 통일시키고 구별하는 것은 가장 기본적인 것이며, 규칙을 일관성 있게 만들어야 프로젝트 진행에서 시행착오와 불필요한 오버헤드를 발생시키지 않게 됨
- 또한 개발자는 다른 문화를 가진 고객과 개발 업체가 소프트웨어 개발 작업 절차를 동일한 관점에서 이해하고 의사소통 과정에서 불편을 없애기 위해서도 표준화 절차는 반드시 필요함

회사마다 다른 작업 절차와 용어의 예

<table>
<tr><th colspan="2">이 책의 단계</th><th>A사</th><th>B사</th><th colspan="2">C사</th></tr>
<tr><td colspan="2">요구 분석</td><td>기획
요구 정의</td><td>기본 설계</td><td colspan="2">요구 정의</td></tr>
<tr><td colspan="2">기본 설계</td><td rowspan="3">개발</td><td>기능 설계</td><td rowspan="2">설계</td><td rowspan="2">처리 설계
화면 설계
클래스 설계</td></tr>
<tr><td colspan="2">상세 설계</td><td>상세 설계</td></tr>
<tr><td rowspan="2">구현</td><td>코딩</td><td>코딩</td><td rowspan="3">구현</td><td>코딩</td></tr>
<tr><td>단위 테스트</td><td rowspan="3">테스트</td><td>단위 테스트</td><td>단위 테스트</td></tr>
<tr><td rowspan="2">테스트</td><td>통합 테스트</td><td>통합 테스트</td><td>통합 테스트</td></tr>
<tr><td>시스템 테스트</td><td>시스템 테스트</td><td>테스트</td><td>시스템 테스트
시스템 운영
테스트</td></tr>
</table>

- 위 표에서 가장 왼쪽 열이 본 강의에서 정의한 개발 절차의 명칭
- 요구 분석, 기본 설계, 상세 설계, 구현으로 나누어지며, 구현의 일부에 단위 테스트가 포함되고 있고, 통합 테스트와 시스템 테스트는 테스트 범주에 속해 있음
  - A사의 경우 요구 분석 작업을 기획과 요구 정의로 구분하고, 설계부터 코딩까지의 작업을 개발로 부르고 있으며, 단위 테스트에서 시스템 테스트까지의 작업을 테스트로 정의
  - B사의 경우 본 강의에서의 요구 분석을 기본 설계로 칭하고, 설계 업무를 기능설계와 상세 설계로 구분
  - C사의 경우 요구분석, 설계, 코딩, 테스트라는 큰 범주를 만들어서 하위 업무를 보다 상세히 구분하고 있지만, 기본 설계와 상세 설계는 구분하지 않고 설계 작업은 외부에서 보이는 부분(처리 설계, 화면 설계)과 내부 구성에 대한 부분(클래스 설계)를 포함
- 이처럼 폭포수 개발 모델을 사용하는 경우에도 작업의 명칭과 이해에 작은 차이를 보이고 있음, 가령 B사의 개발자가 이해하는 기본 설계는 본 강의에서 언급하는 요구분석도 포함하므로 실제 업무에서 차이가 있다고 느낄 수 있음
- 따라서 개발 작업의 명칭에 대한 통일이 필요
- 작업을 일상 업무에서 부를 때 다르게 부르지 않고, 약칭이나 줄임말을 사용하는 경우도 빈번하기 때문에 더욱 중요

- 개발 계획서
  - 프로젝트의 전반적인 방향을 설정하고 성공적인 진행을 위한 구체적인 계획을 수립하기 위해 작성하는 것으로 프로젝트의 목적과 목표, 일정, 예산, 인력, 리스크 관리 계획 등이 포함됨
  - 프로젝트 팀과 이해관계자 간의 의사소통 도구로 활용되며, 프로젝트 성공을 위한 기본 틀을 제공
- 요구 분석서
  - 문제 정의 문서는 현재 상황에서 해결이 필요한 문제를 구체적으로 기술하고, 이를 해결해야 하는 이유를 설명. 이 문서에서는 문제의 배경, 원인, 영향을 체계적으로 분석하며, 이해관계자들이 문제를 명확히 이해할 수 있도록 도움
  - 시스템 제안서는 문제 정의에 기반하여 해당 문제를 해결할 수 있는 시스템을 제안하는 문서로 제안된 시스템의 목표, 주요 기능, 기대 효과, 대략적인 구현 방안 등이 제시하고 프로젝트 착수 전에 이해관계자들에게 시스템의 가치를 설득하는 데 중요한 역할
- 기본 설계서
  - 시스템의 전반적인 구조와 주요 기능을 설계하는 데 초점을 둔 문서로 시스템의 아키텍처, 데이터 흐름, 주요 모듈 및 컴포넌트를 정의하며, 개발팀이 프로젝트를 구체적으로 구현하기 전에 방향성을 명확히 설정하여 전체 시스템의 청사진 역할
- 상세 설계서
  - 기본 설계를 바탕으로 시스템의 세부적인 기술 구현 방안을 문서화한 것으로 알고리즘, 데이터베이스 설계, 인터페이스 명세(API), 화면 설계 등이 포함됨. 개발팀이 실제 코딩 단계에서 참고할 수 있는 구체적인 지침이 되며. 시스템 구현의 일관성과 효율성을 보장하기 위한 핵심
- 프로젝트 종료 보고서
  - 프로젝트 종료 보고서는 프로젝트 완료 후 작성되며, 프로젝트의 성과를 평가하고 주요 교훈을 정리한 것으로, 프로젝트 목표 달성 여부, 결과물, 프로젝트 중 발생한 문제와 해결 방안, 개선점 등이 포함되어 다음 프로젝트를 위한 참고 자료로 활용되며, 프로젝트의 성공 여부와 개선 방향을 평가하는 데 중요한 자료

표는 시스템 개발의 기본적인 문서를 작성하는 시기와, 문서 이름, 문서에 기술해야 할 항목을 나열

같은 내용이라도 요구사항 명세서에 설명하는 기능과 기본 설계서의 기능은 상세도 수준에서 차이가 있음

〈표 2-2〉 프로세스와 산출물

분류	작성 시기	프로젝트 시작	요구 분석		기본 설계	상세 설계	프로젝트 종료
	항목 문서	개발 계획서	문제정의서 (시스템 제안서)	요구 분석 명세서	기본 설계서	상세 설계서	프로젝트 종료 보고서
외부 사양	배경	○	○	○			△
	목적	○	○	○			○
	개요	○	○	○			○
	시스템의 범위	○	○	○			
	기능	○	○	○	○	△	○
	UI		△	○	○		△
	시스템 아키텍처		△	○	○		○
	SW 아키텍처		△	○	○		
	HW 아키텍처		△	○	○		
	네트워크 아키텍처		△	○	○		
	시스템 인터페이스		△	○	○		
내부 사양	모듈 내부 구조					○	
	데이터 구조					○	
	네트워크 구조					○	
	처리 흐름					○	
	메시지 교환					○	
운영	릴리스 전환 계획		○	○			
	운영, 유지 보수		○	○			
공통	용어 정의	○	○	○	○	○	
	표준	○	△	○			○
	품질 관리	○	○	○			○
	비용	○	○	○			○
	산출물	○	○	○			○
	측정						○

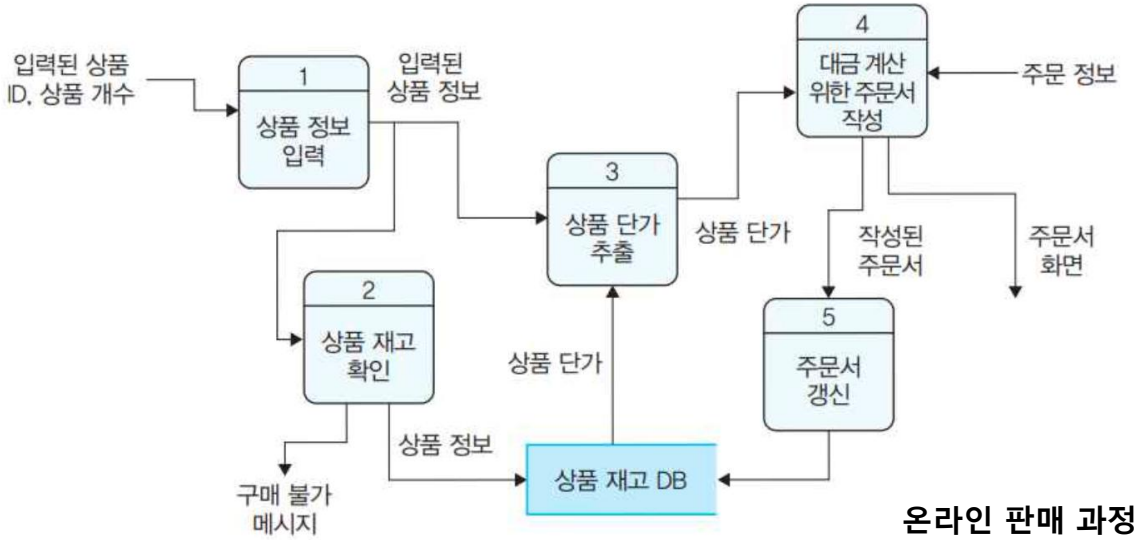
○: 필수 항목, △: 선택적 항목

- 소프트웨어 개발은 프로세스 뿐 아니라 방법론, 즉 개발 방법도 필요로 함.
- 방법을 정확하게 정의하지 않고 프로세스만을 정의하여 혼동하는 경우가 많음
- 방법론은 소프트웨어 프로세스의 각 작업을 어떻게 수행하느냐를 정의한 것으로, 프로세스는 일반적으로 개발할 때 해야 할 일과 산출물을 명시하고 있으며, 방법 자체에 대해서는 다루지 않음
- 방법론은 각 단계에서 요구하는 입력 자료를 가지고 어떻게 작업하고, 그 결과를 표현하는지 알려줌. 방법론에서 제공하는 산출물의 표현 방법은 소프트웨어를 보는 관점에 따라 다름
  - **구조적 방법론**: 시스템을 기능과 프로세스 중심으로 분석하고 설계하는 방법론으로, 모듈화를 통해 복잡성을 줄이고 체계적인 개발을 목표로 함
  - **정보공학 방법론**: 정보 시스템 개발을 기업의 전략 및 비즈니스 목표에 맞춰 계획, 설계, 구축하는 체계적인 방법론으로, 데이터 중심의 설계와 분석을 중시
  - **객체지향 방법론**: 소프트웨어를 데이터와 로직의 통합된 객체로 보고, 데이터를 중심으로 객체와 클래스 개념을 활용하여 시스템을 모델링하고 개발하는 방법론으로, 재사용성과 확장성을 강조

- 구조적 방법론은 1970년대 Tom DeMarco 등이 제안한 이래 많이 사용하는 방법으로 구조적 분석, 구조적 설계 방법으로 구성되어 있음
  - 개발 대상 소프트웨어에 존재하는 처리 개념을 데이터 흐름에 주목하여 계층적으로 상세화 하는 방법으로, 요구분석 단계에서 기능을 정의 → 이를 기초로 설계 단계에서 소프트웨어 구조를 설계
  - 즉 개발할 소프트웨어에 필요한 기능을 명확히 하고, 그 결과를 토대로 소프트웨어의 구조와 처리 방식을 결정

- 구조적 방법론을 이해하기 위한 3개 핵심 내용
  - 자료 흐름도(Data flow diagram, DFD): 그림을 이용하여 소프트웨어에 존재하는 기능들 사이에서 주고 받는 데이터와 그 흐름을 정의
  - 구조도 (Structure chart): 모듈 구조와 모듈 사이의 호출 관계를 정의
  - 소단위 명세서: 모듈 안의 사양을 의사코드로 기술

- 자료 흐름도를 사용하면 개발 대상이 되는 소프트웨어가 갖는 복잡한 기능을 점진적이며 계층적으로 세분화하여 설명할 수 있음
- 이렇게 하면 복잡한 기능을 엉뚱하게 정의하는 위험을 방지할 수 있으며, 적절한 규모의 소프트웨어 명세를 결합하여 전체 시스템의 명세를 설명할 수 있음
- 다시 이를 계층적으로 분할해 나가면 데이터 무결성과 계층 분할의 타당성을 검출할 수 있기 때문에 오류가 적은 기능 명세서를 작성할 수 있음
- 구조 설계는 Edward Yourdon 등에 의해 제안된 방법으로 DFD를 입력 정보로 두고, 모듈의 구조와 모듈 사이의 호출 관계와 실행 순서 등을 정의하는 방법



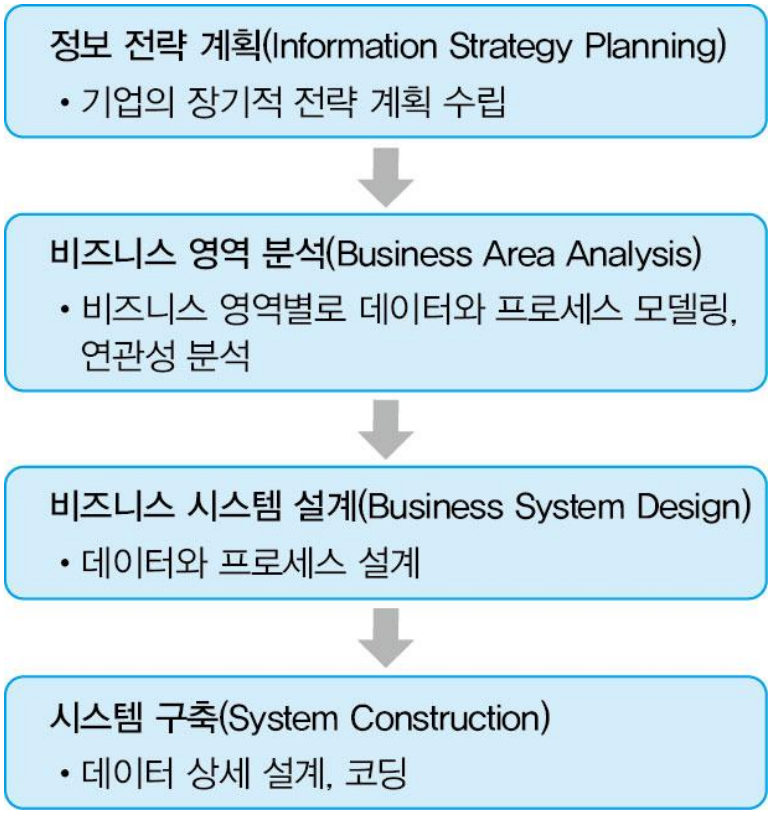
온라인 판매 과정을 표현한 DFD 예

- 구조도를 사용하면 개발 대상 소프트웨어의 구조를 계층적으로 정의할 수 있음
- 또한 구조도를 사용하여 각 모듈의 코드를 작성하고 이를 조합하여 소프트웨어 전체를 구현함
- 온라인 판매 모듈의 구조는 상품 정보 입력 → 상품 재고 확인 → 제품 단가 수출 → 대금 계산 후 주문서 작성 → 상품 재고 갱신 모듈로 이뤄지는 것을 표현

- 이렇게 구조적 분석 설계 기법을 사용하면 개발하고자 하는 소프트웨어에 대한 정보를 도형과 도식으로 나타낼 수 있기 때문에 소프트웨어에 대한 이해를 향상시킬 수 있음
- 또한 다이어그램을 그리는 방법이 표준화 되어 있기 때문에 오해의 소지가 적고 개발 과정에서 생기는 다양한 문제를 줄일 수 있음

- 정보 시스템은 기업의 비즈니스와 밀접하게 관련 되어 있음. 따라서 소프트웨어 프로젝트의 계획, 분석, 설계, 구축 등의 기술 집합이 기업의 부문과 연계/통합하는 방법 필요성 제기
- 이러한 배경에서 1990년대 초 James Martin은 구조적 방법의 문제점, 즉 기업 전체의 조망과 데이터 통합의 어려움을 극복하기 위해 정보공학 방법론을 제안
- 정보공학 방법론의 큰 특징은 다음과 같음
  - 기업 중심: 적용 대상이 기업의 비즈니스 중심으로, 단순한 소프트웨어 개발이 아닌, 기업의 전략경영을 지원하기 위해 전략정보시스템에 초점
  - 전략적 시스템 계획 중심: 시스템의 개발에 경영층의 요구와 견해를 반영하기 위하여 전략적인 계획으로 시작
  - 데이터 중심: 기업의 비즈니스 프로세스는 수시로 변하지만, 데이터는 대부분 변하지 않기 때문에 시스템 유지 보수를 줄이고 잦은 변화에 적극 대응하고자, 데이터 중심으로 설계를 진행
  - 분할과 정복: 비즈니스 문제 영역을 세분화, 즉 분할과 정복의 원리를 적용하여 이해하고 구현함. 즉 톱다운(하향식)방식으로, 문제영역-비즈니스-비즈니스 시스템으로 발전시켜, 최종 구현 시스템이라는 결과물로 완성

- 프로젝트 관점에서 보면, 정보공학 방법론이 요구하는 산출물의 종류는 많지 않음
- 설계 시 스토리보드, 데이터 설계서, 사용자 인터페이스 설계서 정도를 개발하고, 개발 과정에서는 이를 바탕으로 물리적 테이블과 프로그램 코드가 만들어짐
  - 요구 분석에 해당하는 비즈니스 영역 분석 단계에서는 요구 명세서가 만들어짐
  - 시스템 설계 단계에 해당하는 비즈니스 시스템 설계(BSD) 에서는 논리 ERD와 비즈니스 프로세스를 설명하는 프로세스 구조도, 흐름도를 생성함
  - 개발 단계에 해당하는 시스템 구축 단계에서는 물리 ERD, 사용자 인터페이스, 프로그램 코드가 산출됨



- 1990년대 이후 소프트웨어 개발 철학 중 하나로 객체지향 프로그래밍이 큰 화두 – 프로그램이 함수나 데이터가 아닌 객체 중심으로 이뤄진다는 개념은 소프트웨어 패러다임을 크게 바꿈
- 객체는 현실 세계에 있는 물건이나 개념의 특징을 그대로 프로그램화 한 것
  - "텔레비전"의 예
    - 채널, 볼륨, On/off 라는 데이터를 가짐
    - 채널을 돌리다, 볼륨을 변경하다, 켜다/끄다라는 동작이 존재
    - 객체지향은 이러한 방식으로 물건이나 개념을 프로그램화 하는 것
- 객체지향의 기본 개념은 1970년대 사이에 확립되어 있었지만, 1980년에 Smalltalk-80 이라는 객체지향을 도입한 언어가 발표되어 많은 사람들에게 알려짐 → 이후 객체지향 프로그래밍이 도입되고, 객체지향 분석, 객체지향 설계가 탄생하여 현재에 이름

- 객체지향 프로그래밍이 널리 알려지면서 분석 및 설계 단계에서 각종 모델 기법이 다양하게 제안
- UML(Unified modeling language)의 등장
  - 다양한 객체지향 방법론이 논의되는 과정에서 표준화된 표현 방법으로 자리매김
  - UML 1.1 (1997년) 등장
  - UML 2.0 (2004) 발표
- 객체지향 방법론을 사용하여 소프트웨어를 개발할 때 필요한 상위 작업은 다음과 같음
  - 객체지향 분석
    - 소프트웨어 요구사항을 결정하고, 이해하기 위해 유즈 케이스 모델을 구축
    - 유즈 케이스는 사용자와 컴퓨터 시스템 간의 상호 작용을 설명하는 시나리오
    - 응용 프로그램을 구성하는 문제 도메인의 클래스 자체 및 관계 식별도 포함
  - 객체지향 설계
    - 분석 단계에서 기능을 파악하는 도중에 인터페이스, 데이터 액세스, 오퍼레이션을 위해 식별되는 클래스, 속성, 메서드 및 구조를 설계하고 구현하는 작업

3가지 방법론 비교

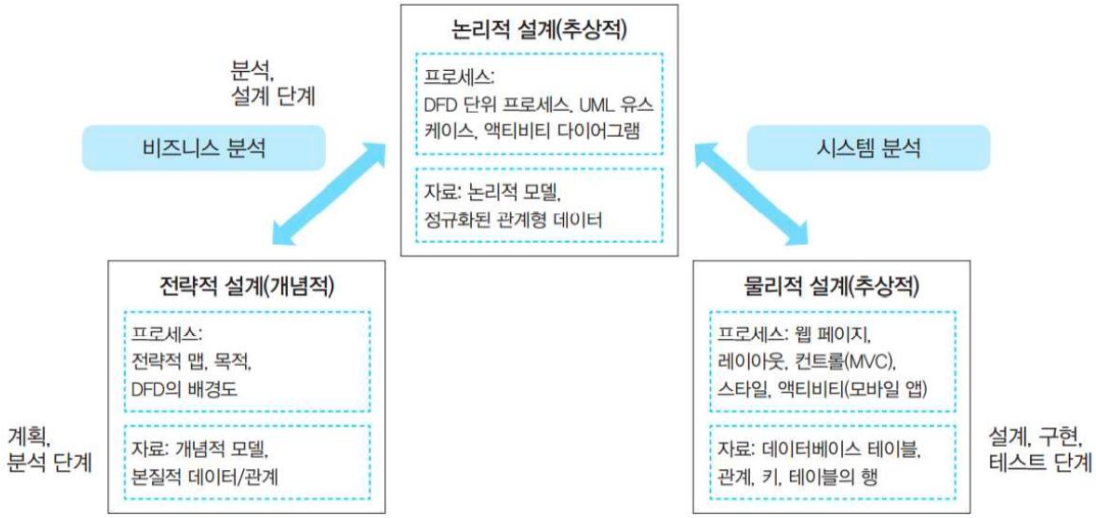
	구조적 방법론	정보공학 방법론	객체지향 방법론
특징	프로그램 로직 중심, 그림(DFD, 구조적 다이어그램) 을 그려 요구사항을 분석하고 문서화하는 체계적 작업	기업 정보 중심이며, 전략 계획을 수립한 후 데이터 중심으로 CASE 도구를 사용, 공학적으로 접근하는 방법	비즈니스를 유즈 케이스로 분석하고, 자료와 함수를 묶은 클래스를 파악하여 상호 작용하는 객체들로 시스템을 구성하는 방법
설계 관심사	함수 위주	자료 위주	클래스 위주
설계의 핵심	모듈	엔티티	객체
중심 방법	프로그래밍 기법	기업의 전략 및 산출물 중심	설계의 표현

	구조적 방법론	정보공학 방법론	객체지향 방법론
장점	• 오래 사용되어 검증된 방법 • 프로젝트 모니터링 용이 • 요구 분석에 대한 방법 이해 쉬움	• 데이터 중심으로 업무 절차 및 환경 변화에 유연함 • 정보 전략 계획, 업무 영역 분석 등 기업에 맞는 접근법	• 실세계와 밀접한 모델링 • 유지보수 용이 • 설계 → 코딩으로의 전환이 쉬움 • 신뢰성, 융통성, 코딩의 재사용성 증가
단점	• 분석에서 설계과정에 대한 전환이 쉽지 않음	• 효과를 얻기 위해 오랜 시간이 필요 • 소규모 자동화 영역에는 시간이 오래 걸림 • 특정 사업 영역으로부터 독립된 시스템 설계에는 부적합	• 광범위한 응용 분야에 효용 증명 되지 못함 • 충분히 훈련된 개발자 부족

- 소프트웨어 시스템을 분석하고 설계할 때 적용되는 일관된 관점 중 하나는 논리적 설계와 물리적 설계의 개념
- 논리적 설계는 시스템의 기능과 각 컴포넌트 사이의 관계를 정의
  - 시스템이 생산해야 할 출력과 준비되어야 할 입력, 시스템이 처리해야 할 프로세스를 정의하는 작업
  - 논리적 설계는 이러한 작업을 컴퓨팅 환경 내에서 어떻게 구현할지는 포함하지 않음
- 물리적 설계는 실제 시스템에 어떻게 구현할지 계획하는 작업
  - 입출력을 구체적으로 화면에 어떻게 보일 것인지, 그 형식과 자료를 파일이나 데이터베이스에 어떻게 보관하고 정렬하고 검출할 것인지 기술

<고객 관리 시스템의 예>

- 논리적 설계 단계에서는 각 고객을 위해 입력할 데이터가 무엇이며 고객 기록을 화면에 어떤 순서로 출력되어야 할지를 결정
- 고객 현황 보고서를 위해 어떤 정보가 생산되어야 할지 고민 후 기술
- 이 때 데이터의 입력 방법, 순서 정렬방법, 생성하는 방법이나 형식은 설계의 대상이 아님



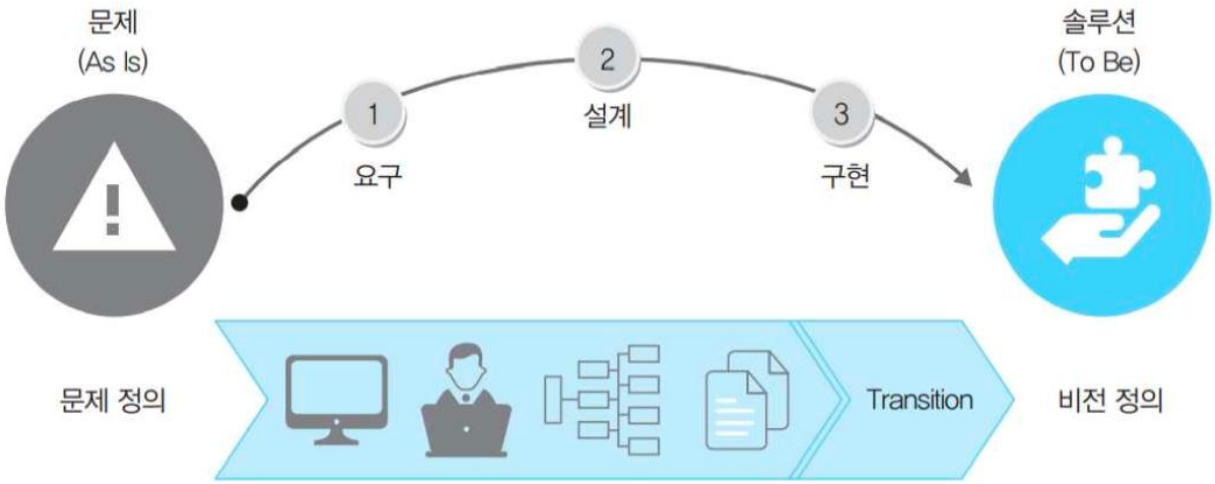
논리적 설계와 물리적 설계

소프트웨어 개발은 매번 새로운 문제를 해결하는 방식

- 개발하려는 소프트웨어가 그 문제를 해결하는 솔루션이 되어야 하기 때문
- 따라서 소프트웨어 개발 단계에서는 현재 시스템의 문제를 잘 정리하여야 하고, 새로운 소프트웨어가 도입되어 어떤 상태로 바뀔 것인지 비전을 보여주어야 함

소프트웨어가 개발되어 업무 프로세스 개선이 이뤄지게 하려면 현재 상태, 즉 as-is 를 파약해야 함

- As-is 란 변경하기나 개선하기 전에 업무를 수행하거나 시스템이 현재 작동하는 방식
- To-be 는 새로운 시스템이 도입되어 예정되어 있는 업무 프로세스

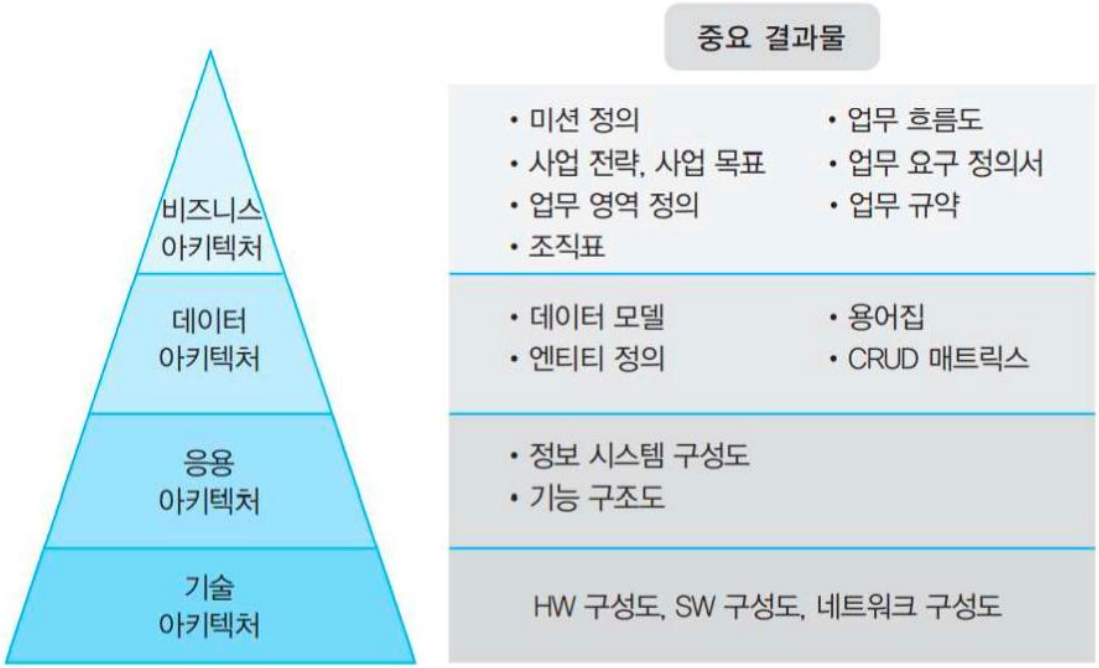


소프트웨어가 개발되어 업무 프로세스 개선이 이뤄지게 하려면 As-is와 To-be를 모두 문서화 하고 매핑해야 하며, 이후 이를 바꾸기 위한 소프트웨어를 설계하고 구현하는 것 자체가 솔루션이 된다고 보면 됨 → 결국 소프트웨어를 개발하려면 현재의 상황이 어떤지 잘 알고 있어야 하며, 이를 분석하고 개선 사항을 제안할 수 있어야 함

소프트웨어를 개발하고 도입하면서 업무의 흐름을 더 효율적으로 개선하는 경우도 있음. 이를 업무 프로세스 개선이라고 하며, 이런 경우 현재 업무 프로세스를 파악하고 정의하고, 이를 개선하거나 최적화 하여 목표 비전이 될 업무 프로세스를 정의해야 함

EA(Enterprise Architecture)

- 정보시스템을 개발하고 소프트웨어를 설계할 때 반드시 알아두어야 할 개념
- 기업 조직이 경영전략을 실행하기 위해 필요한 활동과 컴퓨터 및 소프트웨어자신 전반을 대상으로 함
- 즉 조직의 인력, 정보, 기술에 대한 역할과 목표, 행동, 프로세스, 도구 등을 명확하게 체계화 한 것
- 조직마다 정보화 사업의 기본 구조를 일목요연하게 그려줌



SW를 개발하기 위한 각 작업 사이에 시스템과 관련된 여러 가지 정보를 생성하고 전달할 필요
장황한 글보다는 다이어그램을 많이 사용하며. 그림이 글보다 더 많은 정보를 집약적으로 표현할 수 있음

대표적인 다이어그램

- UML – 객체지향 모델
- 업무 흐름도 – 비즈니스 프로세스
- 순서도 – 프로그램의 처리, 업무 절차, 시스템의 흐름 등
- DFD – 처리 프로세스

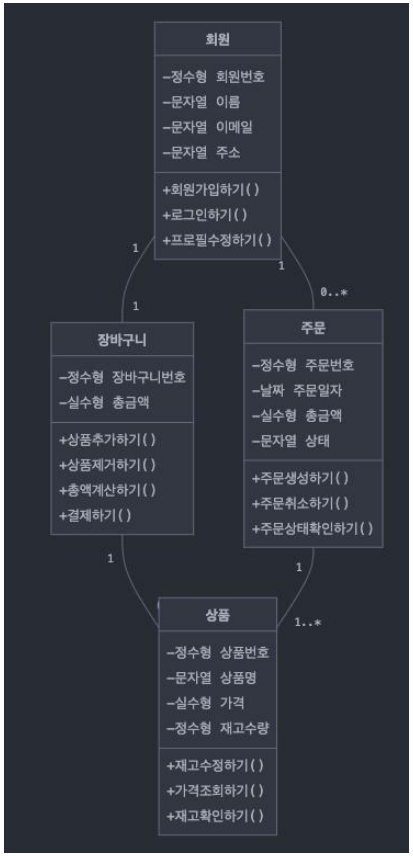
원래 객체지향 개념을 위해 개발되었으나 현재는 일반적인 모델링 언어로 널리 사용되고 있음

표에서 나타난 것 같이 여러가지 다이어그램(문법)이 포함되어 있으며, 구조 다이어 그램과 동적 다이어그램으로 구분할 수 있음

- 정적 – 시간의 흐름과 무관한 상태를 나타내는 정보 (그 자리에 누가 있고 어떤 위치에 있는지?)
- 동적 – 시간의 흐름에 따라 변화하는 것 (누가 누구에게 어떤 질문을 하면 어떻게 응답하는가?)

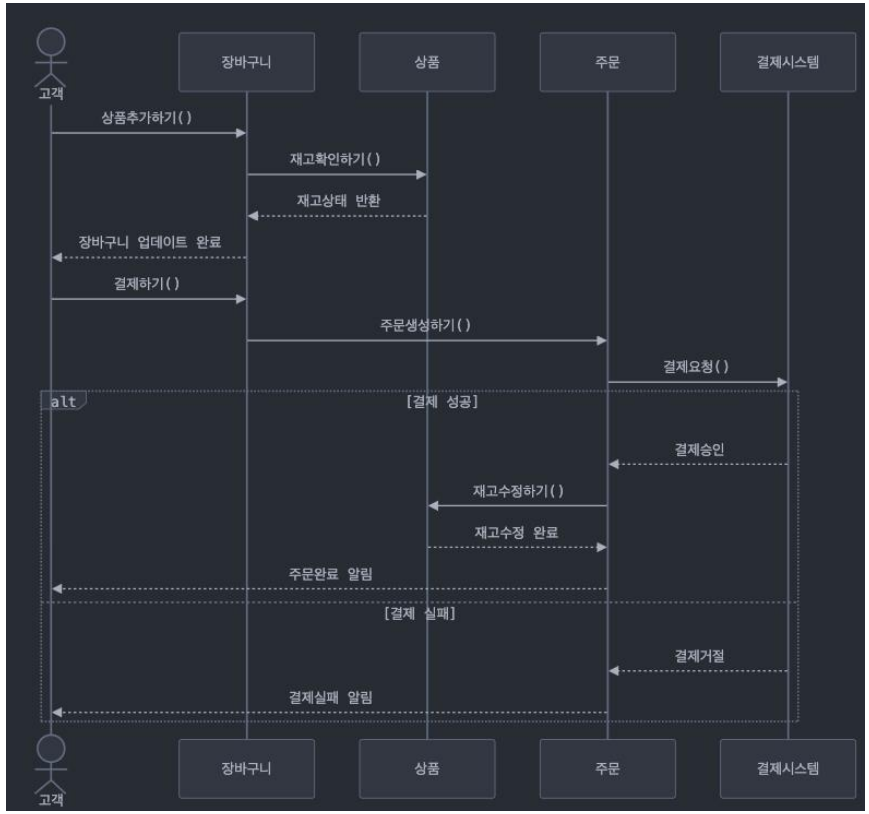
구분	명칭	개요
구조 다이어 그램	클래스 다이어그램	시스템 내부에 어떤 클래스가 존재하며 클래스 사이의 정적 관계를 나타낸다.
	객체 다이어그램	어느 시점에서 시스템에 존재하는 인스턴스의 상태를 나타낸다.
	패키지 다이어그램	개체를 그룹화한 패키지와 패키지 사이의 의존성을 나타낸다.
	배치 다이어그램	하드웨어 또는 소프트웨어의 물리적 위치를 나타낸다.
	컴포넌트 다이어그램	한 시스템에서 사용하는 컴포넌트와 그 관계를 나타낸다.
동적 다이어 그램	시퀀스 다이어그램	객체 사이의 메시지 교환을 시간 흐름에 따라 나타낸다.
	유스 케이스 다이어그램	사용자와 시스템 사이의 일반적인 상호 작용을 나타낸다.
	상태 다이어그램	시스템이나 객체의 상태에 초점을 두어 그 동작을 나타낸다.
	액티비티 다이어그램	처리 로직이나 업무 흐름을 나타낸다.
	커뮤니케이션 다이어그램	객체 사이의 연관성을 나타낸다. 또한 객체 사이에 주고받는 메시지의 교환이 추가된다.
	상호 작용 개요 다이어그램	시퀀스 다이어그램과 액티비티 다이어그램을 결합한 다이어그램이다.
	타이밍 다이어그램	시스템의 동작과 객체의 상태를 시간 흐름에 따라 나타낸 것이며, 전자회로에서 사용되는 타이밍 다이어그램과 유사하다.

유닛4	다이어그램	페이지3	UML



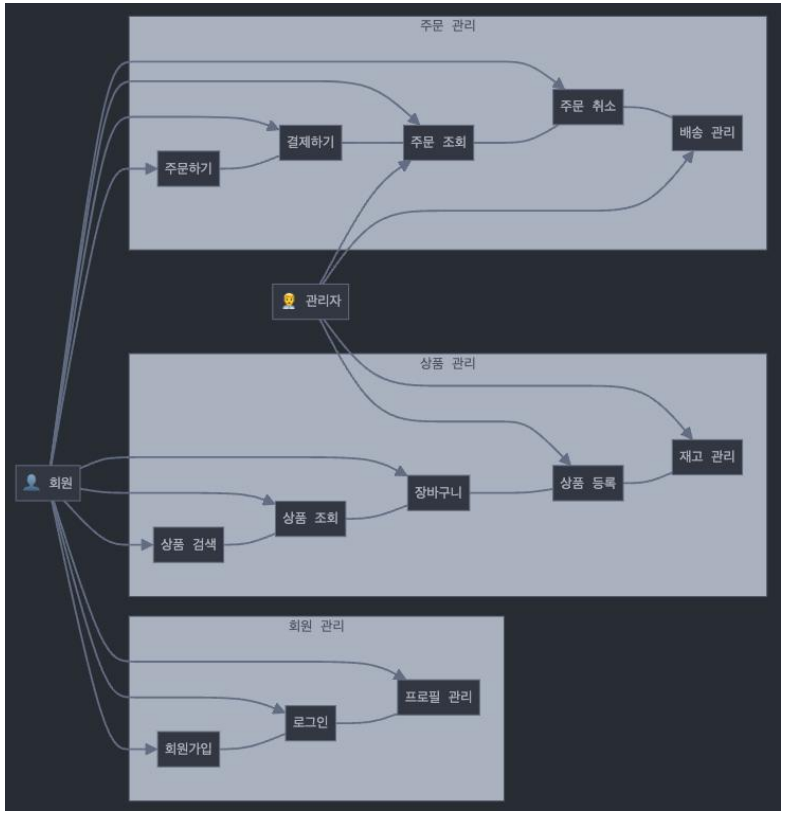
클래스 다이어그램

- 온라인 주문 프로세스를 표현
- 객체인 클래스 사이의 정적인 구성관계를 표현



시퀀스 다이어그램

- 객체 사이의 메시지 교환, 즉 메소드의 호출 관계를 나타냄
- 객체 사이의 동적 상호작용을 시간의 흐름에 따라 표현

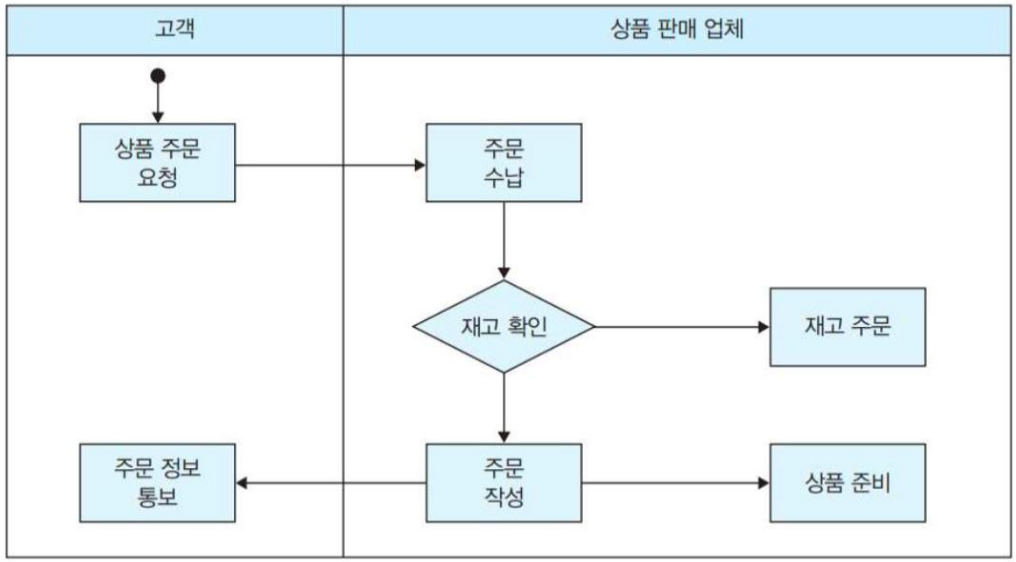


유즈 케이스 다이어그램

- 시스템이 제공하는 기능과 사용자 사이의 동적 관계 표현
- 사용자를 액터라고 지칭하고, 시스템이 제공하는 기능을 유즈 케이스라고 부름

- 소프트웨어 시스템을 개발할 때 As-is 또는 To-be 를 나타내는 방법.
- 업무 흐름도는 알고리즘을 나타내는 흐름도(Flow chart)와 비슷하지만 병렬처리를 나타낼 수 있음
- 프로그램의 논리 흐름보다는 높은 수준의 업무 절차를 나타낼 수 있는 도구이며, UML의 액티비티 다이어그램과 유사
- 소프트웨어 개발 전 업무 흐름을 파악하면 소프트웨어의 기능과 요구를 도출하기 쉬움
- 또한 생산성을 높이고 비용을 줄이기 위한 개선방안을 찾기 위해서 사용되기도 함

업무 흐름도



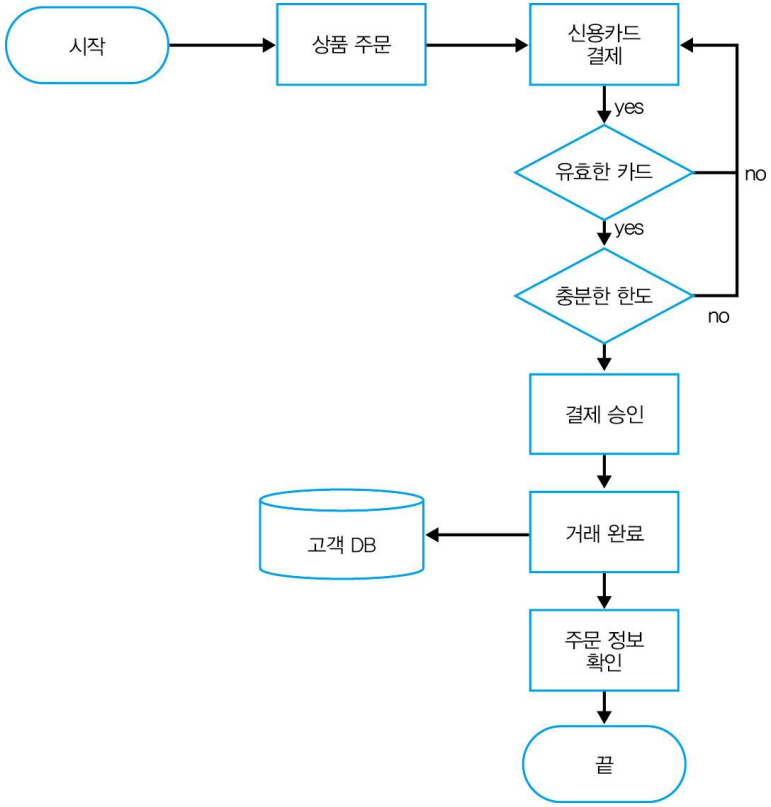
간결한 그림 기호를 사용하여 처리와 데이터의 흐름을 시각적으로 이해하기 쉬운 형태로 그리는 기법
사용하는 기호는 KS와 ISO로 표준화 되어 있음
처리와 데이터의 흐름을 설명하기 위한 기법으로 다년간 사용되어 인지도가 매우 높은 표현 방법
프로그램의 처리 흐름 외에 업무 프로세스와 절차의 흐름을 설명하는 용도로도 사용

순서도는 간단하고 기본적인 기호로 구성되어 있어서 어떤 프로그래밍 언어로 구현하든 널리 이용될 수 있음
그러나 구조적 분석, 객체지향 언어에 필요한 새로운 개념(예: 클래스, 상속 등)을 나타내는 기호가 없다는 점은 한계

순서도에서 사용하는 기호

기호	설명	기호	설명
	처리 '동작 기호'라고도 하며 프로세스, 동작 또는 기능을 나타냄. 순서도에서 가장 널리 사용되는 기호		**터미널** 외부 환경에서의 입구, 출구를 나타내며 프로그램의 시작과 끝을 표시함.
	문서 문서의 입력 또는 출력을 나타냄. 보고서, 이메일, 주문 수신 등의 입력이나 프레젠테이션, 메모, 편지 등의 출력의 생성을 나타냄.		**결정** 조건의 결과(참/거짓)에 따라 다른 분기로 처리되는 것을 나타냄.
	연결자 복잡한 순서도 안에서 주로 사용하는 것으로 별도 페이지의 다른 처리의 흐름으로 연결하라는 의미		**연결선** 데이터나 제어 흐름을 나타냄. 흐르는 방향을 명시할 필요가 있는 경우는 화살표로 그림.
	데이터 입출력을 위하여 준비된 일반 데이터를 나타냄.		**정의된 처리** 서브루틴이나 모듈 등 다른 곳에서 정의된 하나 이상의 연산이나 명령어 집합으로 이루어진 처리
	준비 초기값 설정, 셋업 등 다음의 동작에 영향을 주는 준비 명령 집합을 나타냄.		**데이터베이스** 사용자가 검색하고 필터링할 수 있는 스토리지 서비스에 보관된 데이터를 나타냄.
	수작업 입력 키보드 등 입력 장치를 통해 데이터를 수동으로 입력하는 것을 나타냄.		**표시** 처리 흐름에 정보가 표시될 것을 나타냄.
	수동 연산 자동화되지 않은 수동 작업을 나타냄.		**병합** 여러 개의 경로를 하나로 모음.

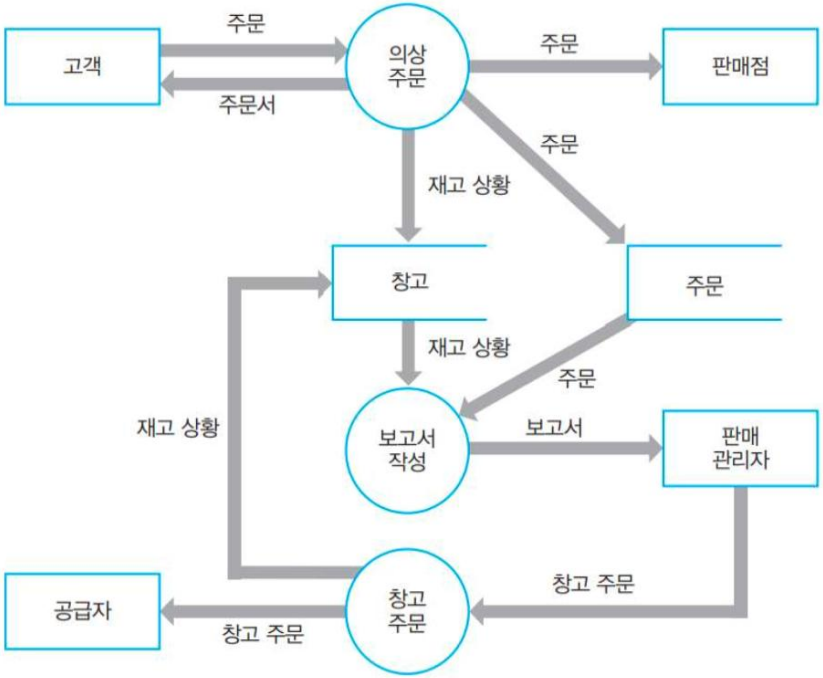
순서도의 예



- DFD(Data Flow Diagram)은 업무나 시스템의 데이터 흐름을 표현하는 기법
- 데이터의 생성, 흐름, 처리를 나타내는데 적합하며, 업무 흐름 이외에도 시스템 내에서의 데이터 흐름을 분명히 하고 분석하는데도 사용
- 그리는 단위를 얼마나 상세하게 하느냐에 따라 자세한 정도를 나타낼 수 있음
- 따라서 우선 일반적인 DFD를 작성하고 세부 내용을 분화하는 방식으로 단계적 설계 가능

심볼	이름	설명
→	데이터 흐름	데이터의 흐름을 나타내는 화살표는 데이터의 출발점에서 목적지를 향해 나간다. 데이터가 다를 때는 다른 화살표로 나타낸다. 흐름도와 같이 처리의 흐름이나 처리를 시작하는 원인을 나타내는 목적으로는 사용하지 않는다.
1.x	프로세스	데이터에 대한 처리를 나타낸다. 프로세스에 들어오는 데이터 흐름을 프로세스에서 나가는 데이터 흐름으로 변환하는 기능을 나타낸다. 프로세스에는 이름과 번호를 붙이고, 프로세스의 이름은 주로 동사로 표현한다.
	저장소	임시 데이터의 위치를 나타낸다. 컴퓨터에 있는 파일이 아니더라도 종이 신청서처럼 업무에 존재하는 파일도 저장소로 표시한다.
	터미널	데이터가 발생되거나 도착되는 위치를 나타내는 기호이다. 즉 DFD의 범위 밖에서 데이터가 들어오는 시작점과 범위를 벗어날 때 데이터가 도착되는 끝점을 나타낸다.

DFD 구성 요소



온라인 주문 사례

학습정리

- 소프트웨어의 각 개발 단계에서는 문서화된 산출물이 생성되며, 이는 프로젝트에 참여하는 모든 이해관계자들에게 명확한 참조가 된다.
- 구조적 방법론, 정보공학 방법론, 객체지향 방법론이 대표적이며 각기 뚜렷한 특징으로 개발 대상이 되는 소프트웨어의 목적, 업무 환경, 개발 환경 등에 따라 선호도가 다를 수 있다.
- 논리적 설계와 물리적 설계 업무는 지향점이 다르기 때문에 상이한 형태의 산출물을 만들어 낸다.
- 다이어그램은 요구사항, 기능, 설계 요소, 관계, 구조 등을 이해하는데 효과적인 도구이며 UML, 업무 흐름도, 순서도, DFD가 보편적으로 사용된다.